HW 1

Isabelle Darling

Georgia Institute of Technology

CS 4675: Computing and Society

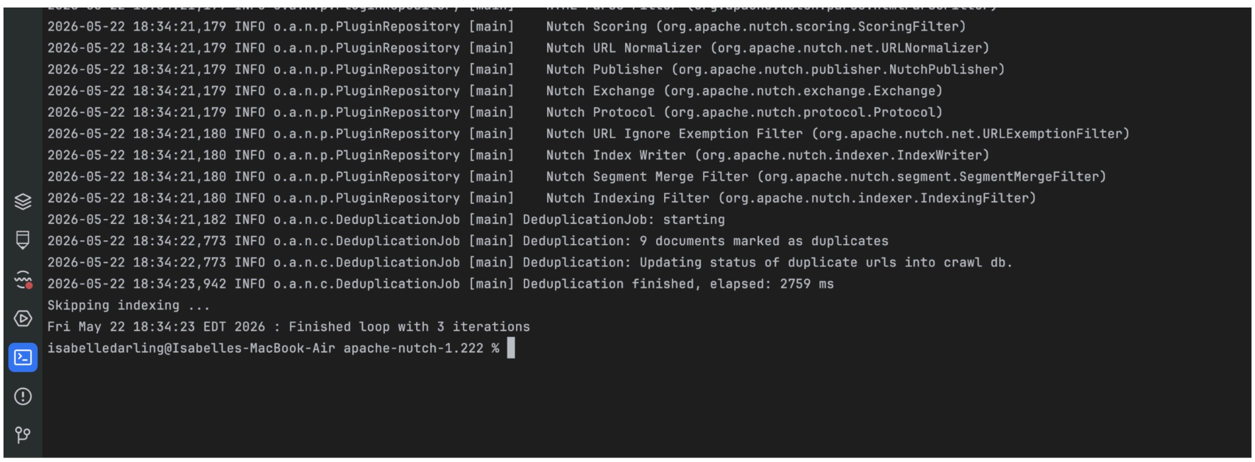
Prof Ling Liu

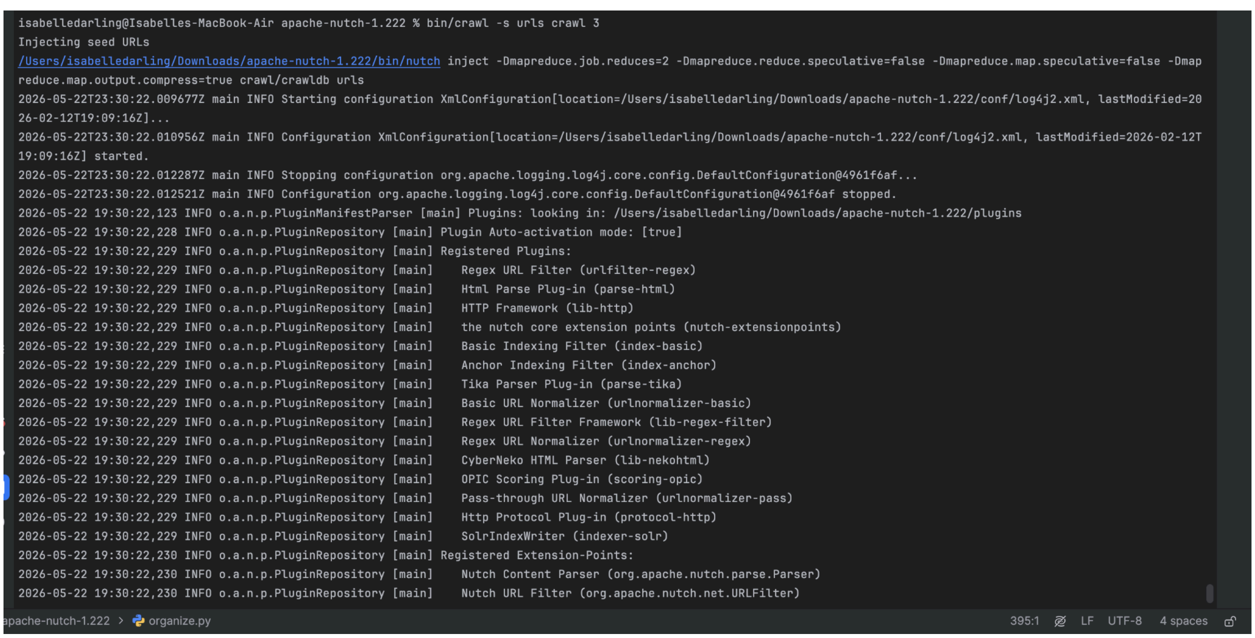
29 May 2026

Problem 1: Option 1.1: Experience with an Open Source Crawlers

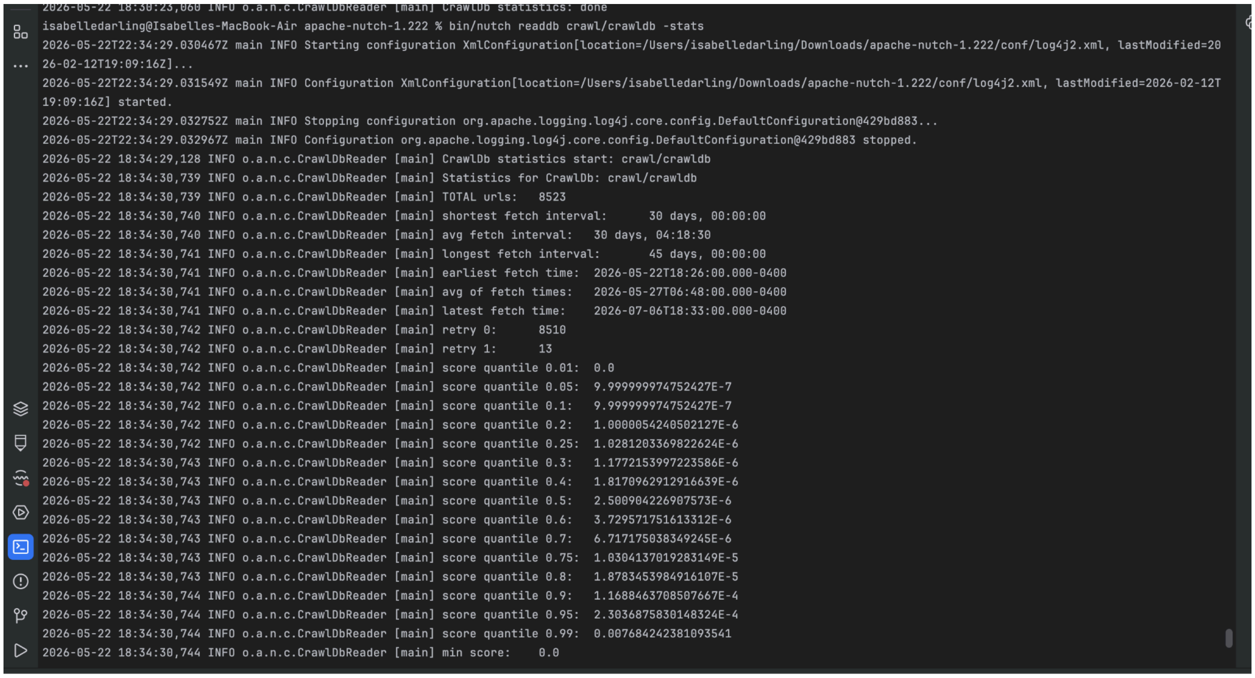
For this assignment, I chose to use the open source crawler, Apache Nutch, from the Apache Lucene website. In order to use this, I chose cc.gatech.edu as the seed by adding in a URLs folder and having a seed.txt file with that seed. I used their built-in tools to crawl with the crawl bash file. I ran 3 iterations of the Nutch crawl, and it ran for approximately 9.7 minutes. I used cc.gatech as the seed.

Running process in the terminal: `bin/crawl -s urls crawl 3`





Below is running the stats with: `bin/nutch readdb crawl/crawldb -stats`



```
2026-05-22 18:30:23,060 INFO o.a.n.c.CrawlDbReader [main] CrawlDb statistics: done
isabelledarling@Isabelles-MacBook-Air apache-nutch-1.222 % bin/nutch readdb crawl/crawldb -stats
2026-05-22T22:34:29.030467Z main INFO Starting configuration XmlConfiguration[location=/Users/isabelledarling/Downloads/apache-nutch-1.222/conf/log4j2.xml, lastModified=2026-02-12T19:09:16Z]...
2026-05-22T22:34:29.031549Z main INFO Configuration XmlConfiguration[location=/Users/isabelledarling/Downloads/apache-nutch-1.222/conf/log4j2.xml, lastModified=2026-02-12T19:09:16Z] started.
2026-05-22T22:34:29.032752Z main INFO Stopping configuration org.apache.logging.log4j.core.config.DefaultConfiguration@429bd883...
2026-05-22T22:34:29.032967Z main INFO Configuration org.apache.logging.log4j.core.config.DefaultConfiguration@429bd883 stopped.
2026-05-22 18:34:29,128 INFO o.a.n.c.CrawlDbReader [main] CrawlDb statistics start: crawl/crawldb
2026-05-22 18:34:30,739 INFO o.a.n.c.CrawlDbReader [main] Statistics for CrawlDb: crawl/crawldb
2026-05-22 18:34:30,739 INFO o.a.n.c.CrawlDbReader [main] TOTAL urls:   8523
2026-05-22 18:34:30,740 INFO o.a.n.c.CrawlDbReader [main] shortest fetch interval:      30 days, 00:00:00
2026-05-22 18:34:30,740 INFO o.a.n.c.CrawlDbReader [main] avg fetch interval:   30 days, 04:18:30
2026-05-22 18:34:30,741 INFO o.a.n.c.CrawlDbReader [main] longest fetch interval:       45 days, 00:00:00
2026-05-22 18:34:30,741 INFO o.a.n.c.CrawlDbReader [main] earliest fetch time:  2026-05-22T18:26:00.000-0400
2026-05-22 18:34:30,741 INFO o.a.n.c.CrawlDbReader [main] avg of fetch times:   2026-05-27T06:48:00.000-0400
2026-05-22 18:34:30,741 INFO o.a.n.c.CrawlDbReader [main] latest fetch time:    2026-07-06T18:33:00.000-0400
2026-05-22 18:34:30,742 INFO o.a.n.c.CrawlDbReader [main] retry 0:      8510
2026-05-22 18:34:30,742 INFO o.a.n.c.CrawlDbReader [main] retry 1:      13
2026-05-22 18:34:30,742 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.01:  0.0
2026-05-22 18:34:30,742 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.05:  9.999999974752427E-7
2026-05-22 18:34:30,742 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.1:   9.999999974752427E-7
2026-05-22 18:34:30,742 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.2:   1.0000054240502127E-6
2026-05-22 18:34:30,742 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.25:  1.0281203369822624E-6
2026-05-22 18:34:30,743 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.3:   1.1772153997223586E-6
2026-05-22 18:34:30,743 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.4:   1.8170962912916639E-6
2026-05-22 18:34:30,743 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.5:   2.500904226907573E-6
2026-05-22 18:34:30,743 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.6:   3.729571751613312E-6
2026-05-22 18:34:30,743 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.7:   6.717175038349245E-6
2026-05-22 18:34:30,743 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.75:  1.0304137019283149E-5
2026-05-22 18:34:30,743 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.8:   1.8783453984916107E-5
2026-05-22 18:34:30,744 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.9:   1.1688463708507667E-4
2026-05-22 18:34:30,744 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.95:  2.3036875830148324E-4
2026-05-22 18:34:30,744 INFO o.a.n.c.CrawlDbReader [main] score quantile 0.99:  0.007684242381093541
2026-05-22 18:34:30,744 INFO o.a.n.c.CrawlDbReader [main] min score:    0.0
```

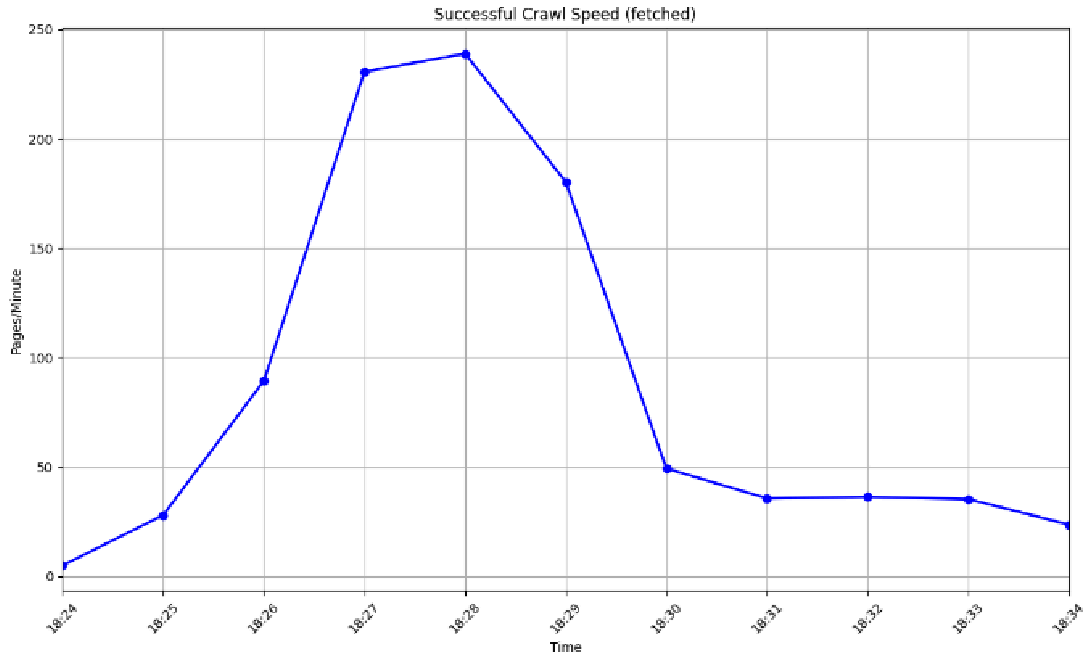
Code and Design:

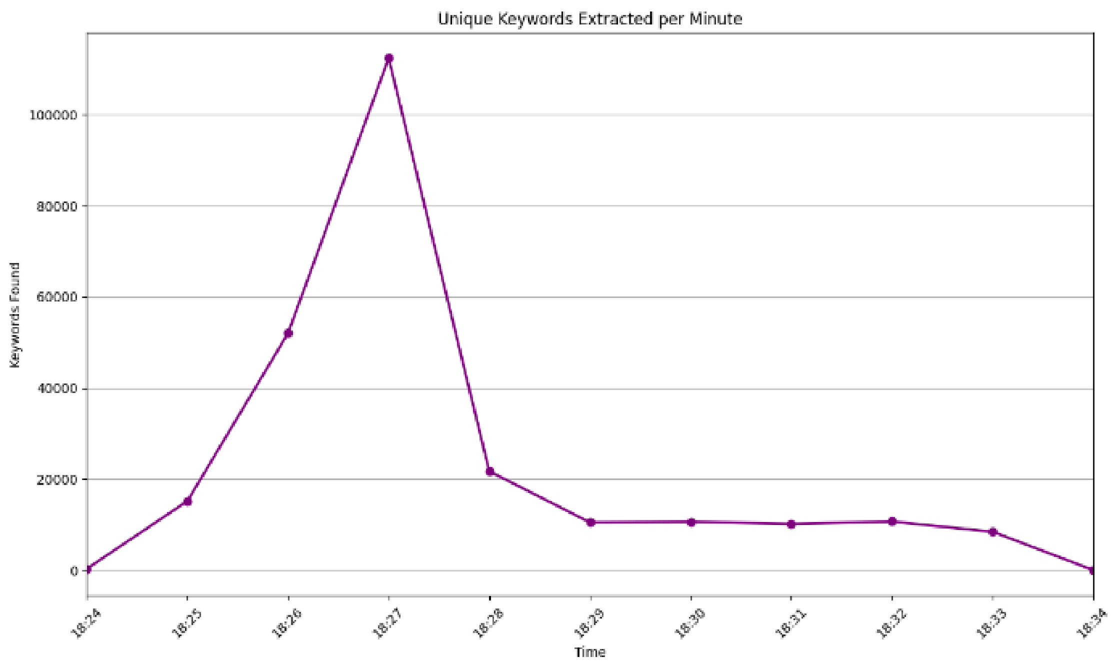
The next part is the analysis. I grabbed the dump file and the segments to analyze crawl speed, general statistics, and keywords extracted. I did this via a Python file: organize.py.

Nutch already creates an archive of the data split into segments (the raw data), linkdb, a database of where all the links link to, and crawdb, a database of all the URLs and fetch data. But since I wanted to analyze this data and pull it in a more organized way, I created a Python script using pandas, re, and matplotlib to take the output from reading the files and graph it. I created the graph itself with matplotlib and read through the data with pandas. The main part is parsing the dump data and throwing it into a CSV file. Using and recording timestamps using the haloop log.

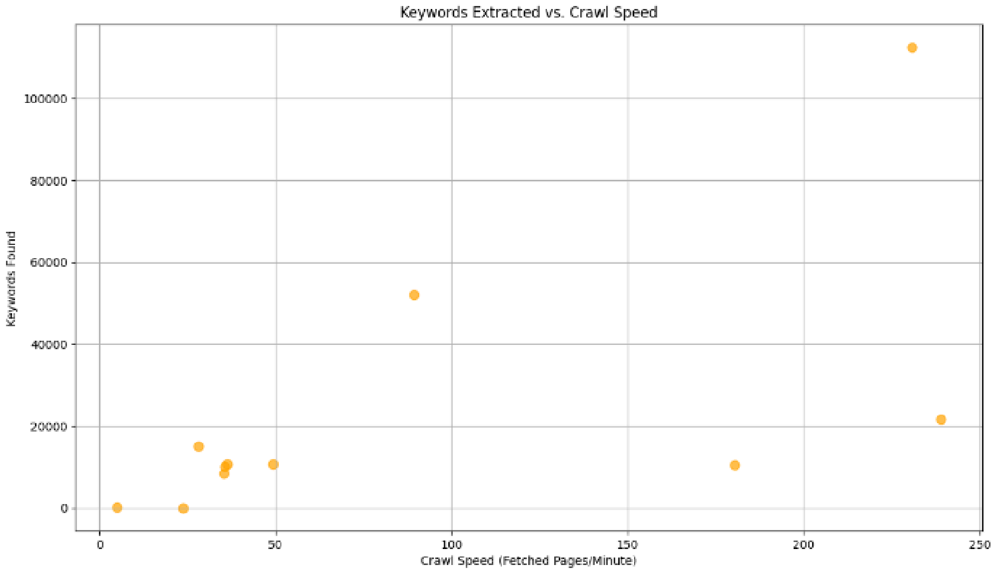
This allowed me to find the crawl speed and map everything according to time. The resulting CSV file has a tab for raw data, analysis stripped straight from the -stats function from Nutch, top unique keywords extracted via nltk, and finally a tab that has all the graphs. I included this all in the source code, but the main changes I made to the code were through running organize.py. To strip the keywords, I created a blacklist of common words like the or technical words in the dump file, like ‘fetch’ or ‘class.’ I imported nltk and downloaded words to check if each parsed keyword was an actual word or just a string of letters. I also created a set so that keywords would be unique.

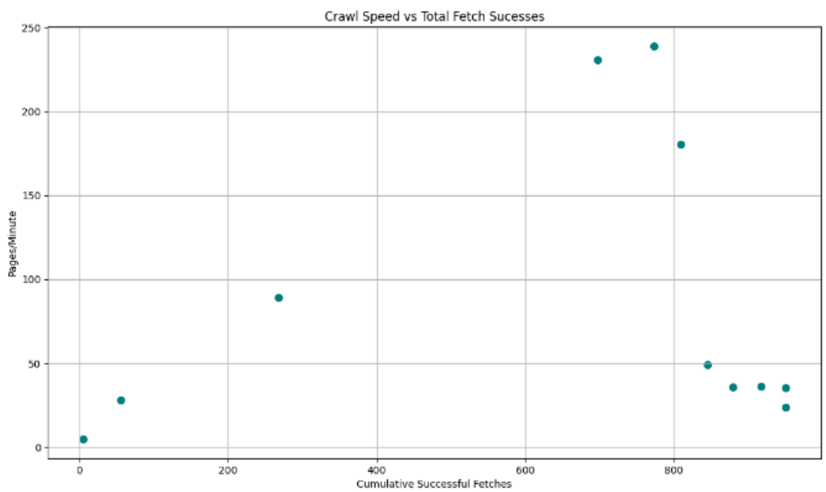
Resulting graphs:

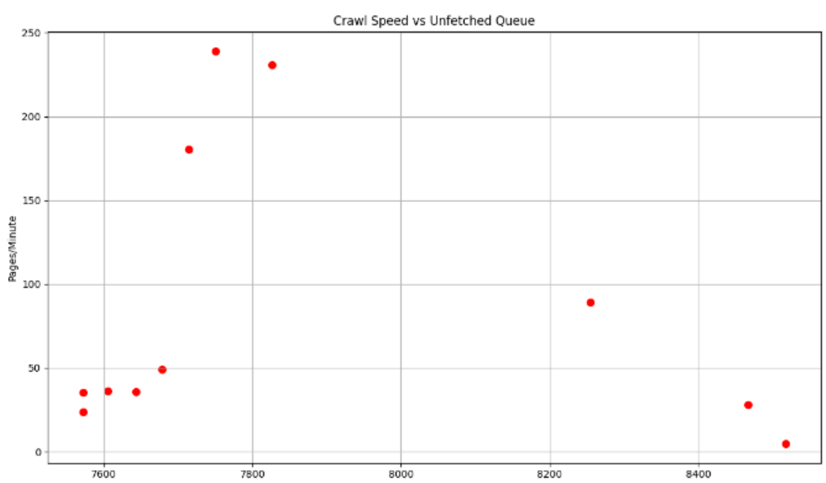




Time


Keywords Extracted vs. Crawl Speed
Keywords Found
100000
80000
60000
40000
20000
0
0
50
100
150
200
250
Crawl Speed (Fetched Pages/Minute)


Crawl Speed vs Total Fetch Sucesses
Pages/Minute
250
200
150
100
50
0
0
200
400
600
800
Cumulative Successful Fetches


Crawl Speed vs Unfetched Queue
Pages/Minute
250
200
150
100
50
0
7600
7800
8000
8200
8400
Unfetched URLs Remaining

Analysis and reflection:

Through problem 1 option 1, I learned how web crawlers work through fetching links from an unfetched queue, indexing them, and storing them in a web archive that can be parsed for data and keywords for search engines to use. I gained experience in specifically using Apache Lucene and Apache Nutch. Looking at the graph, we can see a clear relationship between crawl speed, fetched documents, and a reverse relationship between crawl speed and the size of the unfetched queue. We can also see a positive relationship between unique keywords and crawl speed, which makes sense. Since the crawl speed is calculated as the number of pages fetched divided by the time elapsed, it makes sense that the graph comparing crawl speed and fetched pages would be almost identical. The same is true for the inverse of the unfetched graph size. The graph grows as the crawl speed does, since more pages are being fetched. As the crawler grabs more pages, more unique keywords are found. Notably, there is a large drop-off after 18:28. This is due to the fact that the program counts unique keywords. Common keywords like: Georgia, computing, undergraduate, etc. would not be counted in further pages, which explains the dropoff. There is a relatively positive correlation between keyword extracted and crawl speed, but that changes as more unique keywords are found, and thus fewer new pages result in fewer new keywords.

Here is a list of statistics that can be in the Excel file:

Metric Name	Value
TOTAL URLs	8523
Fetched pages (status 2)	933
Unfetched pages (status 1)	7293
Redirects (status 5)	171
avg fetch time	0
avg fetch interval	30
avg score	3.327822078
min score	0
max score	1.184744
Crawl Speed (pages/minute)	94.56
Ratio (#URL crawled / #URL total)	0.1095

We can estimate our crawler speed for 10 million pages and 1 billion pages. As we can see, crawl speed was at an average of 94.56 with a ratio of about 10% crawled out of all urls. If we assume average crawl speed:

$$\frac{10,000,000}{94.56} = 105752.961083 \simeq 105,753 \text{ minutes}$$

$$\frac{1,000,000,000}{94.56} = 10575296.1083 \simeq 1,575,296 \text{ minutes}$$

But this is only an estimate, as the crawl speed seems to fluctuate wildly. This is quite slow considering how many web pages are on the internet and compared to the speed at which Google web crawlers explore the net.

Through this problem, I primarily learned the process through which websites are crawled. I learned how to set up a crawler, how to seed it, and how to run it through Apache Nutch. It was interesting seeing the top keywords; however, I had some trouble grabbing keywords since there are words that repeat, so I had to create a blacklist for HTML words and words that Nutch used. I learned that it gets exponentially more complex as you increase the iterations of the crawl due to the nature of fetching links.

Additional table of the outputs for the initial core with 110 indexed documents vs the output for the core with 5243 documents: